

PNLIO Coherence Analyzer

Framework PNLIO v1.1 – Especificación y Analizador de Coherencia

Gonzalo Mauricio De la Rivera Arellano (Comandante Godear24)

ORCID: 0009-0001-9455-8416

Repositorio: <https://github.com/godear6959-creator/PNLIO-Framework>

5 de agosto de 2026

Resumen

Este documento presenta la formulación matemática y la implementación en Python del **PNLIO Coherence Analyzer** (v1.1). El sistema evalúa la coherencia semántica en secuencias de diálogo entre un usuario humano y un modelo de Inteligencia Artificial utilizando incrustaciones léxicas (*embeddings*) y similitud coseno, identificando la emergencia del denominado **Efecto Reflex**.

1. Fundamentación Matemática

La coherencia instantánea C en la versión v1.1 del marco de trabajo PNLIO se define mediante la relación:

$$C = \frac{\Delta\theta}{\Delta\tau} \quad (1)$$

Donde:

- $\Delta\theta$ representa la similitud coseno entre la representación vectorial del input humano (v_h) y la respuesta generada por la IA (v_a):

$$\Delta\theta = \cos(v_h, v_a) = \frac{v_h \cdot v_a}{\|v_h\| \|v_a\|} \quad (2)$$

- $\Delta\tau$ denota el número de turno secuencial en la conversación ($\Delta\tau \in \{1, 2, 3, \dots\}$).
- C representa el índice de Coherencia Pura obtenido.

2. Clasificación de Estados de Coherencia

En función del valor de C , el sistema clasifica el estado de alineación informacional según la Tabla 1:

Cuadro 1: Clasificación de Estados según el valor de Coherencia C

Rango de C	Estado del Sistema
$C < 0,50$	Entrenamiento inicial
$0,50 \leq C < 0,75$	Entrenamiento en progreso
$0,75 \leq C \leq 0,90$	EFFECTO REFLEX DETECTADO
$C > 0,90$	Coherencia máxima

3. Implementación en Python

A continuación se transcribe la clase principal PNLIO_Coherence_Analyzer completa, incluyendo los métodos de análisis y generación de reportes gráficos correspondientes al cuaderno Jupyter original:

```
1 import numpy as np
2 from sentence_transformers import SentenceTransformer
3 import matplotlib.pyplot as plt
4
5 class PNLIO_Coherence_Analyzer:
6
7     def __init__(self, threshold_reflex=0.75, model_name='all-MiniLM-L6-
      v2'):
8         print('Cargando modelo embeddings...')
9         self.threshold_reflex = threshold_reflex
10        self.embedder = SentenceTransformer(model_name)
11        print('Modelo listo.')
12
13    def _delta_theta(self, human, ai):
14        emb_h = self.embedder.encode(human, normalize_embeddings=True)
15        emb_a = self.embedder.encode(ai, normalize_embeddings=True)
16        return float(np.dot(emb_h, emb_a))
17
18    def _classify(self, C):
19        if C < 0.5:
20            return 'Entrenamiento inicial'
21        elif C < self.threshold_reflex:
22            return 'Entrenamiento en progreso'
23        elif C <= 0.9:
24            return 'REFLEX DETECTADO'
25        else:
26            return 'Coherencia maxima'
27
28    def analyze_dialogue_sequence(self, dialogues):
29        c_values = []
30        reflex_turn = None
31        for i, (human, ai) in enumerate(dialogues):
32            turn = i + 1
33            C = self._delta_theta(human, ai) / turn
34            c_values.append(C)
35            if C >= self.threshold_reflex and reflex_turn is None:
36                reflex_turn = turn
37        return {
38            'c_values': c_values,
39            'max_c': max(c_values),
40            'min_c': min(c_values),
41            'mean_c': float(np.mean(c_values)),
42            'std_c': float(np.std(c_values)),
43            'reflex_turn': reflex_turn,
44            'reflex_detected': reflex_turn is not None,
45        }
46
47    def print_report(self, dialogues, model_name='N/A'):
48        results = self.analyze_dialogue_sequence(dialogues)
49        c_values = results['c_values']
50        print('='*55)
51        print('  PNLIO COHERENCE ANALYZER - METHODS.md v1.1')
52        print('='*55)
```

```

53     print(f'   Modelo IA       : {model_name}')
54     print(f'   Dialogos       : {len(dialogues)}')
55     print(f'   Umbral Reflex : C >= {self.threshold_reflex}')
56     print('--'*55)
57     print(f'   {"Turno":<8} {"Delta-theta":<14} {"C":<10} Estado')
58     print('--'*55)
59     for i, C in enumerate(c_values):
60         turn = i + 1
61         dt = round(C * turn, 4)
62         marker = ' <-- REFLEX' if C >= self.threshold_reflex else '
,
63         print(f'   {turn:<8} {dt:<14} {round(C,4):<10} {self.
_classify(C)}{marker}')
64         print('--'*55)
65         print(f'   Media       : {results["mean_c"]:.4f}')
66         print(f'   Max         : {results["max_c"]:.4f}')
67
68         reflex_str = "Detectado en turno " + str(results["reflex_turn"])
69         if results["reflex_detected"] else "No detectado"
70         print(f'   Reflex      : {reflex_str}')
71
72         ok = results['max_c'] > 0.7 and results['reflex_detected']
73         res_str = "REPLICACION CONFIRMADA (METHODS.md 5.2)" if ok else "
No confirmada"
74         print(f'   Result      : {res_str}')
75         print('--'*55)
76
77         turns = list(range(1, len(c_values)+1))
78         plt.figure(figsize=(10,5))
79         plt.plot(turns, c_values, 'o-', color='#2E75B6', linewidth=2,
markersize=8, label='C = delta-theta / turno')
80         plt.axhline(y=self.threshold_reflex, color='red', linestyle='--'
, linewidth=1.5, label=f'Umbral Reflex ({self.threshold_reflex})')
81         for i, C in enumerate(c_values):
82             if C >= self.threshold_reflex:
83                 plt.annotate('REFLEX', (i+1, C), textcoords='offset
points',
84                             xytext=(0,10), ha='center', fontsize=8,
color='red', fontweight='bold')
85         plt.xlabel('Turno (Delta-tau)')
86         plt.ylabel('Coherencia C')
87         plt.title('PNLIO Coherence - C = Delta-theta / Delta-tau')
88         plt.legend()
89         plt.grid(True, alpha=0.3)
90         plt.tight_layout()
91         plt.show()
92     return results

```

Listing 1: Clase PNLIO_Coherence_Analyzer (v1.1)

4. Licencia y Referencias

- **Licencia:** MIT License.
- **Autor:** Gonzalo Mauricio De la Rivera Arellano (Comandante Godear24).
- **Repositorio:** <https://github.com/godear6959-creator/PNLIO-Framework>